Yes. For your **Vision Error Observatory (VEO)** work, it is useful to understand monocular depth estimation at two levels:

1. **Classical mathematical geometry** — how depth can theoretically be recovered from a single image using perspective and scene assumptions.
2. **Modern deep-learning monocular depth estimation** — how models such as Depth Anything V2 learn a mapping from an RGB image to a dense depth/disparity map.

Below is the complete mathematical pipeline, from the image formation equation to the depth prediction and evaluation metrics.

---

### 1. What is monocular depth estimation?

Given a single RGB image

$$I \in \mathbb{R}^{H \times W \times 3}$$

the goal is to estimate the depth of every pixel:

$$D \in \mathbb{R}^{H \times W}$$

where

$$D(u, v)$$

represents the distance of the 3D scene point corresponding to image pixel $(u, v)$from the camera.

For example:

RGB Image I



Depth Estimation Model



Depth Map D

Conceptually:

$$\boxed{I(u, v) \rightarrow D(u, v)}$$

The difficult part is that **one 2D image does not uniquely determine a 3D scene**.

---

**2. Why is monocular depth estimation mathematically difficult?**

Suppose a camera observes a point

$$P = (X, Y, Z)$$

in 3D.

The camera projects this point onto the image at

$$(u, v)$$

Multiple different 3D points can project to the same image pixel.

For example:

$$(X, Y, Z) = (1, 1, 5)$$

and

$$(X, Y, Z) = (2, 2, 10)$$

can produce the same image coordinates under perspective projection.

Therefore:

$$\boxed{\text{2D image} \Rightarrow/\text{unique 3D scene}}$$

This is known as the **ill-posed nature of monocular depth estimation**.

A neural network solves this by learning statistical relationships such as:

- perspective
- object size
- texture
- edges
- shadows
- occlusion
- semantic information
- object shapes
- contextual relationships

---

**3. Camera model**

The mathematical foundation starts with the **pinhole camera model**.

Consider a 3D point:

$$P = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

The camera projects it onto the image plane.

The basic perspective equations are:

$$u = f_x \frac{X}{Z} + c_x$$
$$v = f_y \frac{Y}{Z} + c_y$$

where:

- $f_x$= focal length in x direction
- $f_y$= focal length in y direction
- $c_x$= principal point x-coordinate
- $c_y$= principal point y-coordinate

In matrix form:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

The matrix

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

is the **camera intrinsic matrix**.

---

**4. The fundamental depth equation**

From

$$u = f_x \frac{X}{Z} + c_x$$

we can rearrange:

$$u - c_x = f_x \frac{X}{Z}$$

therefore:

$$X = \frac{(u - c_x)Z}{f_x}$$

Similarly:

$$Y = \frac{(v - c_y)Z}{f_y}$$

Therefore, if we know the depth $Z$, we can reconstruct the entire 3D point:

$$\boxed{P = \begin{bmatrix} \frac{(u - c_x)Z}{f_x} \\ \frac{(v - c_y)Z}{f_y} \\ Z \end{bmatrix}}$$

This is extremely important.

It means:

**Depth is the missing variable required to lift a 2D pixel into 3D.**

---

**5. Why depth and disparity are related**

In stereo vision, we have two cameras.

Suppose the cameras are separated by baseline $B$.

A point produces two image coordinates:

$$u_L$$

and

$$u_R$$

The difference is called disparity:

$$d = u_L - u_R$$

Using the camera projection equations, we obtain:

$$\boxed{Z = \frac{fB}{d}}$$

where:

- $Z$= depth

- $f$= focal length
- $B$= baseline
- $d$= disparity

Therefore:

$$Z \propto \frac{1}{d}$$

Large disparity means:

small depth

Small disparity means:

large depth

---

**6. Monocular estimation doesn't directly have disparity**

A single camera doesn't have a second image.

Therefore it cannot directly calculate:

$$d = u_L - u_R$$

Instead, a neural network can learn a representation that behaves similarly to disparity.

This is often called **inverse depth**:

$$\boxed{d \propto \frac{1}{Z}}$$

So many monocular depth models effectively predict something closer to:

$$\hat{d}(u, v)$$

or a normalized depth representation rather than true metric depth.

---

**7. Absolute depth vs relative depth**

This distinction is particularly important for understanding modern models.

Suppose the actual scene depths are:

$$D = \begin{bmatrix} 2 & 4 \\ 6 & 10 \end{bmatrix}$$

Another prediction might be:

$$\widehat{D} = \begin{bmatrix} 20 & 40 \\ 60 & 100 \end{bmatrix}$$

The numerical depths are wrong by a factor of 10.

But the relative ordering is identical:

$$2 < 4 < 6 < 10$$

and

$$20 < 40 < 60 < 100$$

Thus the model has captured **relative depth** even though it has not recovered metric scale.

Mathematically:

$$\widehat{D} = sD$$

where $s$is an unknown scale.

This is called **scale ambiguity**.

---

**8. Scale ambiguity**

Consider the perspective equation:

$$u = f\frac{X}{Z}$$

Suppose we multiply both $X$and $Z$by $s$:

$$X' = sX$$
$$Z' = sZ$$

Then:

$$u' = f\frac{sX}{sZ}$$

which gives:

$$u' = f\frac{X}{Z}$$

Therefore:

$$\boxed{u' = u}$$

The image doesn't change.

So a single image cannot determine absolute scale without additional information.

This is one of the fundamental mathematical limitations of monocular depth estimation.

---

**9. From geometry to machine learning**

Instead of solving depth analytically, modern monocular depth estimation learns a function:

$$f_\theta(I) = \widehat{D}$$

where:

- $I$= input RGB image
- $\theta$= neural network parameters
- $\widehat{D}$= predicted depth

Therefore:

$$\boxed{\widehat{D} = f_\theta(I)}$$

Training attempts to find:

$$\theta^* = \arg\min_\theta L(\theta)$$

where $L$is the depth estimation loss.

---

**10. Neural network representation**

A modern architecture can be conceptually represented as:

$$I \rightarrow E_\theta(I) \rightarrow F \rightarrow D_\phi(F) \rightarrow \widehat{D}$$

where:

- $E_\theta$= encoder
- $F$= learned feature representation
- $D_\phi$= decoder/prediction head
- $\widehat{D}$= predicted depth

The encoder transforms:

$$H \times W \times 3$$

into feature representations such as:

$$\frac{H}{32} \times \frac{W}{32} \times C$$

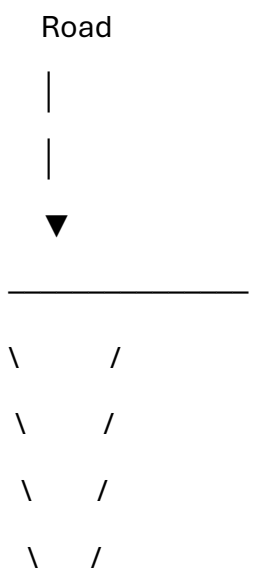
depending on the architecture.

The decoder then converts these high-level features into dense pixel-level predictions.

---

**11. What does the network actually learn?**

Suppose the network sees:




The image contains perspective information.

The network can learn that:

- nearby road pixels tend to appear larger
- distant road regions converge toward the horizon
- objects become smaller with distance
- occlusion boundaries indicate depth discontinuities
- texture changes with distance
- familiar objects have characteristic dimensions

Mathematically, the network approximates:

$$P(D \mid I)$$

or, more simply:

$$D = f_\theta(I)$$

---

**12. Pixel-wise prediction**

For an image with dimensions:

$$H \times W$$

there are:

$$N = H \times W$$

pixels.

For every pixel $i$, the model predicts:

$$\hat{d}_i$$

while the ground truth is:

$$d_i$$

Therefore the prediction problem becomes:

$$\boxed{\hat{d}_i = f_\theta(I)_i}$$

for:

$$i = 1, \dots, N$$

---

**13. Basic depth loss**

The simplest possible loss is Mean Absolute Error:

$$L_{MAE} = \frac{1}{N}\sum_{i=1}^{N} | \hat{d}_i - d_i |$$

Another common loss is Mean Squared Error:

$$L_{MSE} = \frac{1}{N}\sum_{i=1}^{N} (\hat{d}_i - d_i)^2$$

The square makes large errors much more expensive.

For example:

$$e = 2$$

gives:

$$e^2 = 4$$

while:

$$e = 10$$

gives:

$$e^2 = 100$$

---

**14. Why simple pixel-wise loss isn't enough**

Suppose the actual depths are:

$$D = [2, 4, 6, 8]$$

and the prediction is:

$$\widehat{D} = [4, 8, 12, 16]$$

The prediction is exactly:

$$\widehat{D} = 2D$$

The relative structure is perfect, but MAE is large:

$$MAE = \frac{|\,4-2\,| + |\,8-4\,| + |\,12-6\,| + |\,16-8\,|}{4}$$
$$= \frac{2+4+6+8}{4}$$
$$= 5$$

Therefore, if the task is relative depth, raw pixel-wise error may penalize a perfectly scaled prediction.

---

**15. Scale-invariant loss**

One solution is to operate in logarithmic depth space.

Define:

$$g_i = \log \hat{d}_i - \log d_i$$

which is equivalent to:

$$g_i = \log\left(\frac{\hat{d}_i}{d_i}\right)$$

Now consider:

$$\hat{d}_i = sd_i$$

Then:

$$g_i = \log\left(\frac{sd_i}{d_i}\right)$$
$$= \log s$$

Every pixel gets the same error.

The model can therefore distinguish **relative structure** from global scale.

A common scale-invariant formulation is:

$$L_{SI} = \frac{1}{N}\sum_i g_i^2 - \frac{\lambda}{N^2}\left(\sum_i g_i\right)^2$$

The second term removes the global scale component.

---

**16. Why logarithms are useful**

Suppose:

Actual:

$$D = 10$$

Prediction:

$$\widehat{D} = 20$$

The ratio is:

$$\frac{20}{10} = 2$$

Now:

$$\log\frac{20}{10} = \log 2$$

Suppose instead:

Actual:

$$D = 100$$

Prediction:

$$\widehat{D} = 200$$

Again:

$$\log\frac{200}{100} = \log 2$$

So the model sees both as the same **relative error**.

This is useful because depth errors are often better understood proportionally than absolutely.

---

**17. Gradient-based depth information**

Depth boundaries are extremely important.

Consider:

Wall Object



There is a depth discontinuity at the object boundary.

Mathematically, depth gradients are:

$$\nabla D = \begin{bmatrix} \frac{\partial D}{\partial x} \\ \frac{\partial D}{\partial y} \end{bmatrix}$$

The magnitude is:

$$|\nabla D| = \sqrt{\left(\frac{\partial D}{\partial x}\right)^2 + \left(\frac{\partial D}{\partial y}\right)^2}$$

Large values indicate strong depth changes.

---

**18. Connection to your VEO project**

This is particularly relevant to your **Vision Error Observatory** project.

You are investigating how depth estimation fails based on image characteristics such as:

- edge density

- texture
- brightness
- contrast
- depth discontinuities

These can be mathematically connected to the depth prediction process.

For example:

$$\text{Edge Density} = \frac{\text{number of edge pixels}}{\text{total pixels}}$$

You can compare this with depth error:

$$E_{depth} = \frac{1}{N}\sum_{i} | \widehat{D}_i - D_i |$$

and investigate whether:

$$E_{depth} = f(\text{edge density})$$

or more generally:

$$E_{depth} = f(E_d, T, B, C, G_d)$$

where:

- $E_d$= edge density
- $T$= texture
- $B$= brightness
- $C$= contrast
- $G_d$= depth discontinuity information

---

**19. Texture mathematically**

Suppose an image patch contains pixel intensities:

$$x_1, x_2, \dots, x_N$$

Mean intensity:

$$\mu = \frac{1}{N}\sum_{i} x_i$$

Variance:

$$\sigma^2 = \frac{1}{N}\sum_i (x_i - \mu)^2$$

Standard deviation:

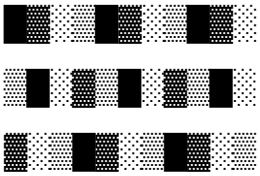
$$\sigma = \sqrt{\sigma^2}$$

A high local variance generally indicates more texture.

A low variance region could be:



while high variance might be:



The relationship between texture and depth error can then be experimentally studied.

---

**20. Image gradients**

For image intensity $I(x, y)$:

$$I_x = \frac{\partial I}{\partial x}$$
$$I_y = \frac{\partial I}{\partial y}$$

Gradient magnitude:

$$G = \sqrt{I_x^2 + I_y^2}$$

An edge detector essentially identifies pixels where:

$$G > T$$

for some threshold $T$.

Edge density can then be approximated as:

$$ED = \frac{1}{N}\sum_{i=1}^{N} 1\,(G_i > T)$$

This gives you a numerical feature that can be compared against depth-estimation error.

---

**21. Brightness**

For an RGB pixel:

$$(R, G, B)$$

a common luminance approximation is:

$$Y = 0.299R + 0.587G + 0.114B$$

Average image brightness:

$$B = \frac{1}{N}\sum_{i} Y_i$$

Very dark or very bright images can reduce useful visual information.

This can potentially affect:

$$\widehat{D} = f_\theta(I)$$

and therefore:

$$E = f\left(\widehat{D}, D\right)$$

---

**22. Contrast**

One simple measure of contrast is standard deviation:

$$C = \sqrt{\frac{1}{N}\sum_{i} (Y_i - \mu_Y)^2}$$

Higher $C$ means greater intensity variation.

More sophisticated definitions use local contrast, RMS contrast, or Michelson contrast depending on the image characteristics.

---

**23. Depth discontinuity**

Suppose ground-truth depth is:

$$D(x, y)$$

A depth discontinuity can be detected through:

$$| \nabla D(x, y) |$$

Large:

$$| \nabla D |$$

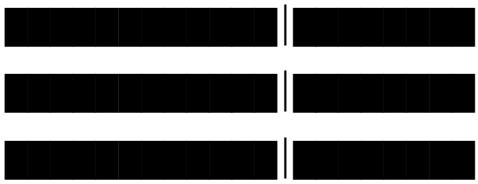
means depth changes rapidly.

These regions are often difficult because a small pixel shift can produce a large depth error.

For example:

Background Foreground

20m 2m



↑

depth boundary

---

**24. Error metrics**

Once the model predicts:

$$\widehat{D}$$

we compare it with:

$$D$$

using several metrics.

**MAE**

$$\boxed{MAE = \frac{1}{N} \sum_{i} | \hat{d}_i - d_i |}$$

Measures average absolute error.

---

**RMSE**

$$\boxed{RMSE = \sqrt{\frac{1}{N}\sum_{i}(\hat{d}_i - d_i)^2}}$$

RMSE heavily penalizes large errors.

---

**Relative Absolute Error / AbsRel**

$$\boxed{AbsRel = \frac{1}{N}\sum_{i}\frac{|\hat{d}_i - d_i|}{d_i}}$$

This measures error relative to actual depth.

Example:

Actual:

$$d = 10$$

Prediction:

$$\hat{d} = 12$$

then:

$$\frac{|12 - 10|}{10} = 0.2$$

or 20%.

---

**25. Squared Relative Error**

Another metric is:

$$SqRel = \frac{1}{N}\sum_{i}\frac{(\hat{d}_i - d_i)^2}{d_i}$$

This penalizes larger errors more strongly.

---

**26. Accuracy under threshold**

A very common depth-estimation metric uses:

$$\delta_i = \max\left(\frac{\hat{d}_i}{d_i}, \frac{d_i}{\hat{d}_i}\right)$$

Then calculate:

$$\delta < 1.25$$

The percentage of pixels satisfying this is:

$$\delta_1 = \frac{\#\{i: \delta_i < 1.25\}}{N}$$

Similarly:

$$\delta_2 : \delta_i < 1.25^2$$

and

$$\delta_3 : \delta_i < 1.25^3$$

Higher percentages indicate more predictions within those multiplicative error bounds.

---

**27. Why the δ metric is interesting**

Suppose:

$$d = 10,\ \hat{d} = 12$$

Then:

$$\frac{12}{10} = 1.2$$

So:

$$\delta = 1.2 < 1.25$$

The prediction qualifies.

Now suppose:

$$d = 10,\ \hat{d} = 20$$

Then:

$$\delta = 2$$

so it fails the $1.25$threshold.

This makes δ metrics naturally suited to **relative depth accuracy**.

---

**28. The complete training mathematical process**

The entire deep-learning process can be expressed as:

**Step 1 — Input**

$$I \in \mathbb{R}^{H \times W \times 3}$$

**Step 2 — Feature extraction**

$$F = E_\theta(I)$$

**Step 3 — Depth prediction**

$$\widehat{D} = D_\phi(F)$$

Therefore:

$$\widehat{D} = f_{\theta,\phi}(I)$$

**Step 4 — Calculate error**

For example:

$$L = \frac{1}{N}\sum_i | \hat{d}_i - d_i |$$

**Step 5 — Backpropagation**

Calculate:

$$\nabla_\theta L$$

and:

$$\nabla_\phi L$$

**Step 6 — Update parameters**

Using gradient descent:

$$\theta_{new} = \theta - \eta \nabla_\theta L$$
$$\phi_{new} = \phi - \eta \nabla_\phi L$$

where:

$$\eta$$

is the learning rate.

This process repeats for many training examples.

---

**29. What happens during backpropagation?**

Suppose the final prediction is:

$$\widehat{D} = f_\theta(I)$$

and loss is:

$$L\left(\widehat{D}, D\right)$$

We want:

$$\frac{\partial L}{\partial \theta}$$

Using the chain rule:

$$\frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial \widehat{D}} \frac{\partial \widehat{D}}{\partial \theta}$$

The network then modifies its weights so that the next prediction should reduce the loss.

Repeated over millions of pixels and many images:

$$\boxed{\theta \to \theta^*}$$

where $\theta^*$is the learned parameter set.

---

**30. Why pretrained monocular models work without your training**

This is important for your VEO implementation.

When you use a pretrained depth model, you aren't mathematically starting from:

$$\theta_0$$

and training it on your dataset.

Instead, you already have:

$$\theta^*$$

learned from large-scale training.

Your process becomes:

$$I \to f_{\theta^*}(I) \to \widehat{D}$$

You then analyze:

$$\widehat{D}$$

rather than training the model yourself.

That is exactly why pretrained models are useful for your failure-analysis project.

**31. Relative depth vs metric depth in modern models**

Suppose ground truth:

$$D = [2, 5, 10, 20]$$

and model:

$$\widehat{D} = [0.2, 0.5, 1, 2]$$

Then:

$$\widehat{D} = 0.1D$$

The model has essentially recovered the structure but not the metric scale.

You can align the prediction:

$$D' = s\widehat{D}$$

where:

$$s = \frac{D}{\widehat{D}}$$

in the simplest conceptual case.

More generally, scale alignment can be estimated by least squares.

We minimize:

$$L(s) = \sum_i (s\hat{d}_i - d_i)^2$$

Taking derivative:

$$\frac{dL}{ds} = 2\sum_i \hat{d}_i \left(s\hat{d}_i - d_i\right)$$

Set equal to zero:

$$\sum_i \hat{d}_i \left(s\hat{d}_i - d_i\right) = 0$$

Therefore:

$$s\sum_i \hat{d}_i^2 = \sum_i \hat{d}_i\, d_i$$

and:

$$\boxed{s = \frac{\sum_i \hat{d}_i \, d_i}{\sum_i \hat{d}_i^2}}$$

This gives the optimal scale factor under least-squares scale alignment.

---

**32. Scale-and-shift alignment**

Some monocular predictions may require both scale and shift:

$$D' = s\widehat{D} + t$$

We minimize:

$$L(s,t) = \sum_i (\, s\hat{d}_i + t - d_i)^2$$

This is essentially a linear regression problem.

The solution gives the scale $s$and shift $t$that best align prediction and ground truth.

This is particularly useful when evaluating relative-depth models.

---

**33. Why edges can cause errors**

Imagine:

Background

20 m

[redacted]

[redacted]

[redacted]

[redacted] Object

[redacted] 2 m

The actual depth changes abruptly:

$$20 \rightarrow 2$$

So:

$$|\, \nabla D \,| \gg 0$$

If the predicted boundary is shifted by only 2 pixels, several pixels can have extremely large errors.

Therefore:

$$\boxed{\text{Boundary localization error} \rightarrow \text{large depth error}}$$

This is one reason your VEO analysis of **depth discontinuities** is meaningful.

---

**34. Why texture can cause errors**

Consider a textureless wall:

$$I(x, y) \approx C$$

Therefore:

$$\nabla I \approx 0$$

There is little local information telling the model about surface orientation or distance.

Compare that with a textured surface:

$$\nabla I \neq 0$$

The network has more visual cues.

However, high texture does **not automatically mean better depth**.

Repeated textures can be ambiguous.

For example:

| | | | | | | |

| | | | | | | |

| | | | | | | |

The model may have difficulty determining whether the pattern represents:

- nearby small objects
- distant large objects
- repeated structures

So your analysis should investigate the relationship empirically rather than assuming:

$$\text{more texture} = \text{better depth}$$

---

**35. Why shadows can create errors**

Image intensity is not determined only by geometry.

A simplified image formation model is:

$$I(x,y) \approx R(x,y)\, L(x,y)$$

where:

- $R$= surface reflectance
- $L$= illumination

Two surfaces at the same depth can have different brightness:

$$D_1 = D_2$$

but:

$$I_1 \neq I_2$$

Conversely, two regions with different depth may have similar brightness.

Therefore:

$$I \leftrightarrow /D$$

in a deterministic way.

This is one of the reasons monocular depth estimation is fundamentally ambiguous.

---

**36. Why semantic information helps**

Suppose the model sees:



/ \

/ \

----------

Road

The model has learned typical relationships involving cars, roads, buildings, people, etc.

So its prediction can effectively use:

$$P(D \mid I)$$

rather than relying purely on local pixel values.

This is a key advantage of deep learning.

---

**37. Probabilistic interpretation**

We can think of depth estimation as estimating:

$$P(D \mid I)$$

Using Bayes' rule:

$$P(D \mid I) = \frac{P(I \mid D)P(D)}{P(I)}$$

where:

- $P(D \mid I)$= probability of depth given image
- $P(I \mid D)$= likelihood of image given depth
- $P(D)$= prior knowledge about possible scenes
- $P(I)$= normalization term

A neural network learns a complex approximation to this relationship from data.

This explains why the model can infer depth even when direct geometric information is missing.

---

**38. A complete mathematical picture**

The entire process can therefore be summarized as:

$$\boxed{I \xrightarrow{\text{camera}} \text{2D projection of 3D world}}$$

The inverse problem is:

$$\boxed{I \xrightarrow{f_\theta} \widehat{D}}$$

Training:

$$\boxed{\theta^* = \arg\min_\theta L(f_\theta, (I)D)}$$

Prediction:

$$\boxed{\widehat{D} = f_{\theta^*}(I)}$$

Evaluation:

$$\boxed{E = M(\widehat{D}, D)}$$

where $M$ can be:

$$MAE, RMSE, AbsRel, SqRel, \delta$$

And your VEO failure analysis then investigates:

$$\boxed{E = F(\text{texture, edges, brightness, contrast, depth discontinuities, ...})}$$

---

**39. The most important equations for your viva**

If your panel asks you to explain the mathematical foundation, these are the equations I'd memorize.

**Camera projection**

$$\boxed{u = f_x \frac{X}{Z} + c_x}$$

$$\boxed{v = f_y \frac{Y}{Z} + c_y}$$

**3D reconstruction**

$$\boxed{X = \frac{(u - c_x)Z}{f_x}}$$

$$\boxed{Y = \frac{(v - c_y)Z}{f_y}}$$

**Stereo depth relationship**

$$\boxed{Z = \frac{fB}{d}}$$

**Neural depth prediction**

$$\boxed{\widehat{D} = f_\theta(I)}$$

**Optimization**

$$\boxed{\theta^* = \arg\min_\theta L}$$

**MAE**

$$\boxed{MAE = \frac{1}{N}\sum_i |\,\hat{d}_i - d_i\,|}$$

**RMSE**

$$RMSE = \sqrt{\frac{1}{N}\sum_{i}(\hat{d}_i - d_i)^2}$$

**AbsRel**

$$AbsRel = \frac{1}{N}\sum_{i}\frac{|\hat{d}_i - d_i|}{d_i}$$

**δ accuracy**

$$\delta_i = \max\left(\frac{\hat{d}_i}{d_i}, \frac{d_i}{\hat{d}_i}\right)$$

**Depth gradient**

$$|\nabla D| = \sqrt{D_x^2 + D_y^2}$$

---

**40. The key idea to tell your VEO panel**

If they ask:

**"What is the mathematical problem that your project is studying?"**

A strong answer is:

**Monocular depth estimation is an ill-posed inverse problem. A 3D scene is projected onto a 2D image through perspective projection, losing information about absolute depth and scale. A deep-learning model learns an approximation $\widehat{D} = f_\theta(I)$ from visual cues such as perspective, texture, edges, semantics and occlusion. Our project studies where this learned mapping produces errors and whether those errors correlate with measurable image and scene characteristics such as edge density, local texture variance, brightness, contrast and depth discontinuities.**

That connects the **camera geometry → neural network → depth prediction → error metrics → VEO failure analysis** into one mathematical story.